DPTO. ELECTRICIDAD Y ELECTRÓNICA

TPMMEE - 2021/22

CONTROL DE ALTAVOZ CON SENSOR DE CHOQUE

AUTOR: Lucas Crespo

06/05/2022

Hoja de Guarda

TABLA DE CONTENIDOS

1. **Objetivos 4**
2. **Conceptos esenciales necesarios para la práctica 4**
3. **Interpretación de la ejecución del programa 5**

PRÁCTICA DE IMPLEMENTACIÓN DE SENSORES ARDUINO

TPMMEE - 2021/22

TPMMEE - 2021/22

PRÁCTICA 6

——— Lucas Crespo

1. Objetivos.

Se diseñará un programa empleando al menos uno de los sensores del kit Arduino estudiado en clase. El sensor elegido es el shock sensor o sensor de choque, el cual se empleará en un circuito que demuestra el funcionamiento de otro periférico: un lector de tarjetas SD.

Más concretamente, el programa usará un lector de tarjetas SD de arduino, el cual se usa para reproducir música por un altavoz de 8 ohms, usando un AO LM386N para amplificar la señal de sonido. El circuito montado reproducirá pistas de audio cuando se pulse un botón designado para ello o se detecte un choque o golpe al circuito.

2. Conceptos esenciales necesarios para la práctica.

Antes de realizar la implementación del programa, repasamos algunas funciones internas de Arduino y conceptos de C++ que nos ayudarán con la implementación del código.

Considerando sensores y demás periféricos que NO han sido explicados en informes anteriores:

- *pinMode():* La función pinMode() establece la funcionalidad de un pin digital como entrada o salida. Hasta ahora se han empleado INPUT y OUTPUT para lograr esta funcionalidad, pero en esta práctica se emplea un botón que no mantiene su estado al ser pulsado. Para realizar lecturas correctas de este componente se emplea la asignación INPUT_PULLUP.

- *digitalRead()*: La función digitalRead() permite a un pin de entrada digital detectar un estado lógico alto o bajo.

- *Comparadores lógicos And y Or:* Su sintaxis es *&&* y || respectivamente.

En cuanto a los sensores y periféricos , su funcionamiento individual es el siguiente:

Shock sensor: Un sensor muy sencillo que consiste en un interruptor físico que actúa en presencia de una colisión. Se conecta VCC y GND y a un tercer terminal a un pin de lectura digital para detectar su estado. Extraer un estado lógico bajo del sensor equivale a detectar una colisión.

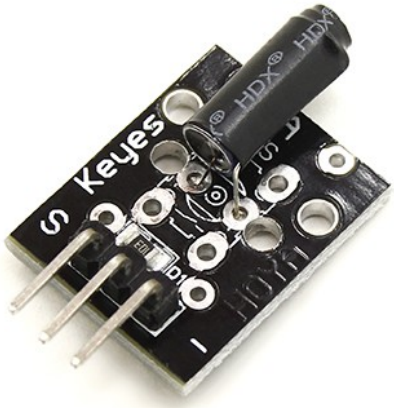

Figura 1: Shock sensor.

Lector de tarjetas SD: Se conecta VCC y GND. Tiene tres terminales más que por defecto (establecido en las librerías empleadas) conectamos a los pines digitales 11 (MOSI), 12 (MISO) y 13 (SCK). Adicionalmente, tiene un cuarto pin CS será conectado a una entrada que declaremos nosotros.

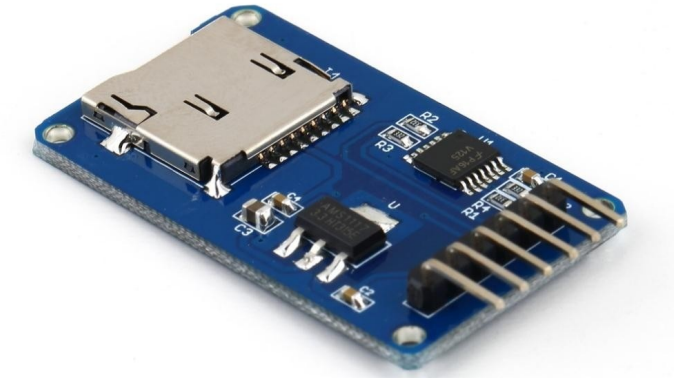

Figura 2: Lector de tarjetas SD.

Para programar el lector de tarjetas con idea de reproducir archivos de audio se usan diferentes librerías: SD (Librería para leer la tarjeta SD), TMRpcm (Librería para reproducir audio en formato .wav a través de la creación de un objeto como si de un servomotor se tratase) y SPI.h(Librería de comunicación con periféricos). De estas tres librerías, las funciones más relevantes son: *music.speakerPin = N* como pin de salida de audio y *music.play("nombre.wav").*

Por último cabe destacar que la señal de audio emitida por el arduino es demasiado baja. Para poder escucharla el montaje emplea un amplificador operacional LM386N, conectado a un altavoz de 8 ohms. Sin entrar en detalle del circuito, el esquema de conexiones se ve de la siguiente manera:

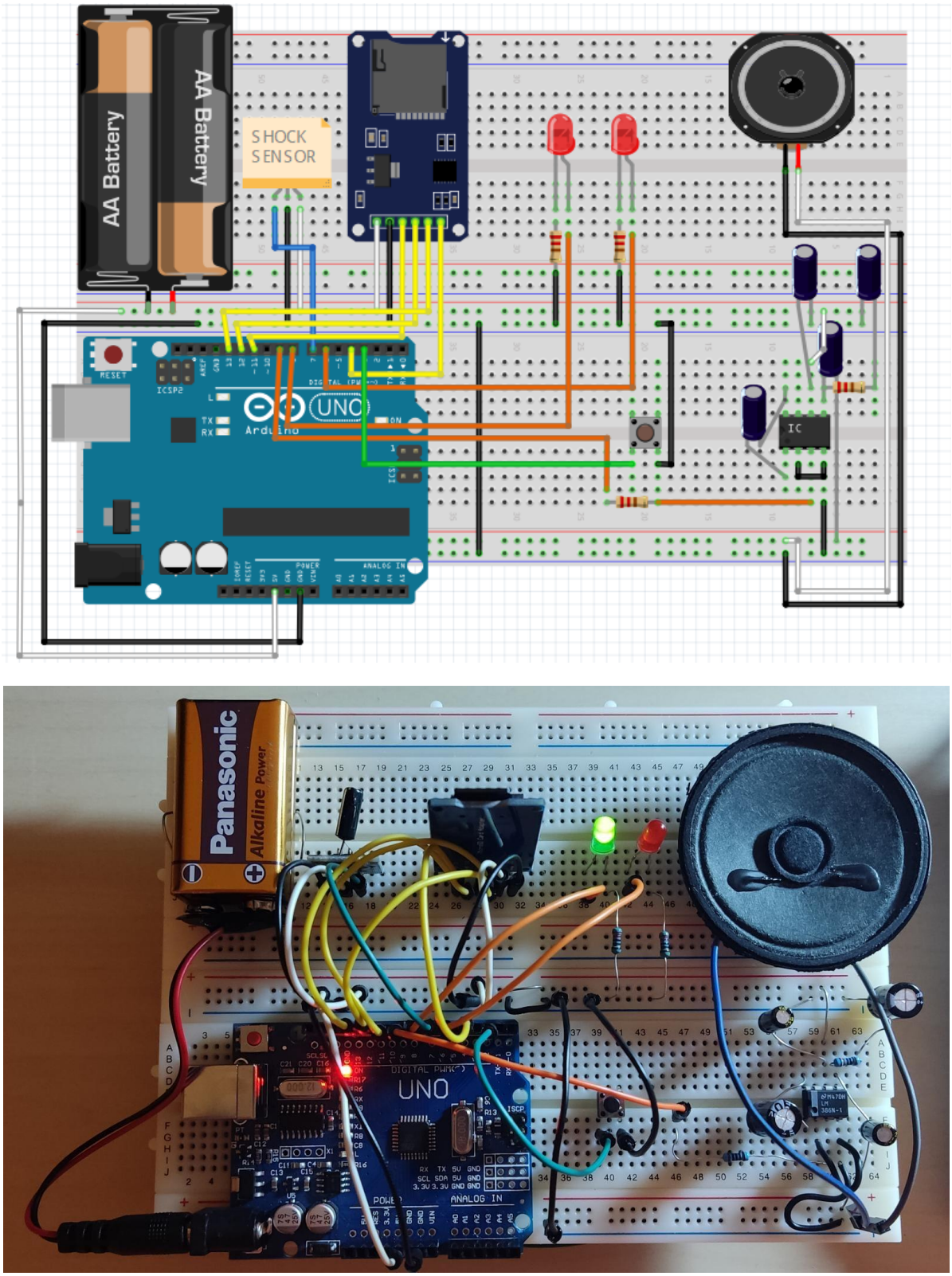

Figura 3: Esquema de conexiones y montaje sobre la placa protoboard.

3. Interpretación de la ejecución del programa.

El programa comienza importando las librerías necesarias descritas en el apartado anterior:

```
#include "SD.h"      //Librería de lectura de la tarjeta SD
#include "TMRpcm.h" //Librería para reproducir audio .wav
#include "SPI.h"     //Libreria de comunicación con periféricos
```

En void setup() se establecen las condiciones iniciales del programa, comenzando por la declaración de pines digitales y otros parámetros de ejecución. Vemos que se reservan pines para LEDs a parte de los de lectura de estado del sensor de choque y botón.

```
pinMode(INIT_LED, OUTPUT);
pinMode(ERROR_LED, OUTPUT);
pinMode(SHK_PIN, INPUT);
pinMode(BOTON_PIN, INPUT_PULLUP);
```

```
music.speakerPin = MUSIC_PIN;
music.setVolume(VOLUMEN);
music.quality(CALIDAD_SONIDO);
```

Antes de continuar, se ejecuta una vez (en el setup()) una condición de detección de errores: comprueba que el lector de tarjetas está conectado correctamente y enciende un LED verde si está correcto y lo mantiene durante el resto de ejecución; o enciende un LED rojo e impide continuar con la ejecución:

```
if (!SD.begin(SD_PIN)) {            // Comprobar que se inicializó el lector de tarjetas
  fallo = true;
  Serial.println("SD fail");
} else {Serial.println("SD init"); digitalWrite(INIT_LED, HIGH); digitalWrite(ERROR_LED, LOW);}
```

Al entrar en el bucle principal comenzamos incluyendo un delay que evita que el sensor de choque se sature a lecturas,

```
delay(PAUSA_SHK);
```

y se comprueban condiciones para reproducir audio:

- ☐ Si el lector de tarjetas está correctamente inicializado y se detecta que el botón fue pulsado (LOW porque está conectado a tierra), o se detecta un choque

```
if (digitalRead(BOTON_PIN) == LOW || digitalRead(SHK_PIN) == LOW && fallo == false)
```

se entra en la rutina de reproducción de música: un delay evita saturación de lecturas, de las condiciones iniciales, un contador permite que cada vez que se entre en la rutina se reproduzca la canción siguiente (volviendo a empezar de llegar al final), y reproduciendo la canción correspondiente. El nombre de las cancione es el definido en la tarjeta SD:

```
delay(PAUSA_SD);
NUMERO_CANCION ++;
if (NUMERO_CANCION == 4) {NUMERO_CANCION = 1;}

Serial.println("Boton pulsado");
Serial.print("NUMERO_CANCION=");
Serial.println(NUMERO_CANCION);

if (NUMERO_CANCION == 1){music.play("1.wav");}
if (NUMERO_CANCION == 2){music.play("2.wav");}
if (NUMERO_CANCION == 3){music.play("3.wav");}
```

- ☐ Si el lector de tarjetas no está correctamente inicializado, se enciende el LED rojo

```
else if (fallo == true){

  digitalWrite(INIT_LED, LOW);
  digitalWrite(ERROR_LED, HIGH);
```